

PROMPTS — SPRINT FINAL HotelEquip CRM OS

Data: 15/07/2026 · Branch: `feat/modulo-propostas` · Autor: Claude (consultoria) para Rui Medalha

PROTOCOLO DE USO (ler antes de colar o primeiro prompt)

1. **Um prompt de cada vez.** Só passas ao seguinte depois de a verificação do anterior estar verde.
2. **Ordem obrigatória:** P0 → P1 → P2 → P3 → P4 → P5 → P6 → P7.
3. Cada prompt obriga o Claude Code a: ler o CLAUDE.md → planear → implementar → **testar com dados reais** → commit + push → **apresentar evidências**.
4. **Nunca aceites "concluído" sem:** (a) hash de commit verificável no GitHub, (b) outputs reais dos comandos de verificação, (c) critérios de aceitação cumpridos. Se faltar um, responde: "Mostra as evidências do protocolo de entrega."
5. Verificação independente: no fim de cada prompt, cola-me aqui o relatório do Claude Code e eu confirmo por API (GitHub + Directus + produção) antes de avançares.

REGRAS GLOBAIS (aplicam-se a todos os prompts):

-  **NEWSLETTER É INTOCÁVEL:** proibido ler para envio, escrever, apagar ou testar com `newsletter_subscriptions` e com os campos `newsletter_*`, `coupon_*`, `consentimento`, `subscribed_at` de `contacts`. Nenhum teste pode tocar nestes dados.
 -  **Envios de teste (email):** SÓ para `ruimedalha@hotelequip.pt`. Nenhum outro destinatário, nunca.
 -  **Envios de teste (WhatsApp/SMS):** SÓ para `916542271`. Nenhum outro número, nunca.
 -  Todos os registos de teste criados levam prefixo `[TESTE]` no título/nome e são listados no relatório final para limpeza.
 -  Não refactorar código que funciona. Não "melhorar" nada fora do pedido. Um problema de cada vez.
 -  Antes de qualquer alteração em massa de dados: export JSON de backup para `/var/backups/crm/` com timestamp, e o caminho do ficheiro no relatório.
-

DECISÃO DE ARQUITETURA — A LEAD É UMA ENTIDADE ATIVA (fixada 15/07 com o Rui)

A lead não é um registo fino: é uma ficha de trabalho completa até ser promovida.

1. **Povoada por completo na ingestão** (não ao abrir): nome, empresa, telefone da assinatura, NIF, morada/cidade/CP, website, produtos pedidos (`lead_data.itens` + `sku_history`), urgência/prazo (`commercial_notes`). Os campos já existem na coleção `leads` — é enchê-los.
2. **Com timeline**: emails, chamadas Telecof e conversas WhatsApp ligam-se à lead (`lead_id`) e aparecem na ficha da lead.
3. **Com follow-ups** próprios, visíveis na Agenda.
4. **Cria proposta — e criar proposta promove automaticamente a lead a contacto** (fluxo de promoção existente), com badge visível "Convertida de lead". Cadeia sempre limpa: lead → contacto → deal → proposta.
5. Nota de futuro (fora deste sprint): `contacts` já tem `entity_type`/`entity_status` — a convergência para entidade única fica para depois da estabilização. Não migrar agora.

PROMPT 0 — Preparar o CLAUDE.md para o sprint (5 min)

Lê o CLAUDE.md na raiz do repositório. Estamos na branch feat/modulo-propostas.

TAREFA ÚNICA: acrescenta ao CLAUDE.md uma secção nova no topo chamada "SPRINT FINAL — REGRAS ATIVAS (15/07/2026)" com exatamente este conteúdo:

1. NEWSLETTER INTOCÁVEL: proibido qualquer operação (leitura para envio, escrita, teste) sobre a coleção `newsletter_subscriptions` e sobre os campos `newsletter_*`, `coupon_*`, `consentimento` e `subscribed_at` de `contacts`.
2. TESTES REAIS: emails de teste só para `ruimedalha@hotelequip.pt`; WhatsApp/SMS de teste só para 916542271. Nenhum outro destinatário/número, sem exceções.
3. Registos de teste levam prefixo [TESTE] e são listados no relatório para limpeza.
4. Schema Directus: alterações SEMPRE via `docker exec db-hotelequip psql` (nunca `PATCH /fields`), seguidas de `POST /utils/cache/clear` e 3 segundos de espera.
5. Meilisearch: `https://search.palamenta.com.pt`, índice `products_palamenta`, chamado DIRETAMENTE via `fetch` (nunca via proxy Directus). Campos: `title`, `thumbnail`, `images[]`, `brand`, `sku`, `ean`, `categories[]`, `short_description`, `price`,

stock_status, url.

6. IA: sempre via proxy n8n <https://n8n.hotelequip.pt/webhook/ai-proxy> (token hotelequip-ai-2026). NUNCA chaves no frontend.

7. Entrega de cada tarefa: hash de commit real + push + outputs dos comandos de verificação. Sem evidências, a tarefa não está concluída.

8. Não refactorar código que funciona; não tocar em nada fora do âmbito do pedido.

Depois: git add CLAUDE.md, commit com mensagem "chore: regras do sprint final no CLAUDE.md", push para feat/modulo-propostas.

ENTREGA: hash do commit + output de `git log -1 --oneline`.

PROMPT 1 — A triagem comanda o fluxo (emails deixam de virar lixo)

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07):

- A classificação de emails JÁ funciona: email_threads.category é preenchido (valores observados: pedido_orcamento, reclamacao, fatura_administrativo, tabela_precos_fornecedor, fornecedor_sourcing, compra_cliente, spam) e ai_summary também.
- PROBLEMA: a classificação não comanda nada. Emails de fornecedores (ex.: edenox) e newsletters viram leads na mesma. 64 leads de email existem, muitas são fornecedores/spam.
- A criação de leads a partir de email acontece no pipeline de ingestão (descobre onde: procura no repositório e nos workflows n8n o ponto onde email → lead é criado; documenta o que encontrares antes de mexer).

OBJETIVO: a categoria decide o destino do email. NENHUM email de fornecedor, fatura ou spam volta a criar lead.

REGRAS DE ENCAMINHAMENTO A IMPLEMENTAR:

- pedido_orcamento, compra_cliente → cria/atualiza LEAD (comportamento atual, mantém)
- reclamacao → cria lead com tag "pos-venda" e prioridade alta (para já; módulo próprio virá depois)
- tabela_precos_fornecedor, fornecedor_sourcing → NÃO cria lead; marca a thread com status "sourcing" (fila de compras)
- NOVA CATEGORIA fornecedor_novidade → NÃO cria lead; marca thread com status

"novidade" (fila de conteúdo/blog). Atualiza o prompt do classificador para distinguir: novidade de produto/newsletter de fornecedor conhecido = fornecedor_novidade; tentativa de venda fria = fornecedor_sourcing.
- spam → NÃO cria lead; thread arquivada (status "arquivado")
- fatura_administrativo → NÃO cria lead; mantém thread com status próprio "administrativo"

TAREFAS:

1. Documenta o fluxo atual (onde nasce a lead de email: workflow n8n ID + nós, ou código no repo). Escreve o que descobriste em docs/fluxo-email.md.
2. Implementa o gate de encaminhamento no ponto certo do fluxo (n8n ou backend – usa o que já existe, não crie um sistema paralelo).
3. POVOAMENTO COMPLETO DA LEAD NA INGESTÃO (não ao abrir): quando a categoria manda criar lead, extrai via proxy ai do corpo+assinatura: nome da pessoa, empresa, telefone, NIF, morada/cidade/código-postal, website, produtos mencionados e urgência/prazo. Preenche os campos próprios da lead que JÁ EXISTEM: contact_person, contact_phone, contact_email, nif, city, postal_code, website, display_name; produtos → lead_data.itens e sku_history; urgência/prazo → commercial_notes. A extração ao abrir (existente) mantém-se como fallback, sem duplicar leads (usa dedupe_key).
4. Atualiza o prompt do classificador (via proxy ai) para incluir fornecedor_novidade, com 2 exemplos.
5. LIMPEZA HISTÓRICA (com backup primeiro para /var/backups/crm/): nas leads existentes de source email/email_inbound cujas threads têm category de fornecedor/spam/fatura, marca status=rejected com note "triagem automática: <categoria>". NÃO apagues nada. Reporta quantas foram marcadas.

TESTES REAIS OBRIGATÓRIOS (nenhum email sai para fora de ruimedalha@hotelequip.pt):

- T1. Envia via Microsoft Graph, de ruimedalha@hotelequip.pt para a caixa ingerida pelo CRM, um email [TESTE] com texto de pedido de orçamento COM assinatura completa (nome, empresa, telefone 262 000 000, morada, website) e produtos no corpo ("2 vitrinas refrigeradas e 1 forno de convecção"). Espera o ciclo de ingestão. VERIFICA por API: thread com category=pedido_orcamento E lead criada COM contact_person, contact_phone, city, website preenchidos E lead_data.itens com os 2 produtos.
- T2. Envia email [TESTE] com texto de newsletter de fornecedor ("Novidade: nova gama de fornos..."). VERIFICA: category=fornecedor_novidade, status=novidade, e NENHUMA lead criada.
- T3. Confirma por query que o total de leads NÃO aumentou com T2.

CRITÉRIOS DE ACEITAÇÃO (mostra o output real de cada um):

- A1. curl às threads de hoje agrupadas por category+status mostra o encaminhamento correto.

A2. curl às leads criadas hoje: zero com origem em categorias de fornecedor/spam/fatura após o deploy.

A3. Query à lead do T1: contact_person, contact_phone, city, website e lead_data.itens preenchidos (mostra o registo).

A4. docs/fluxo-email.md existe no repo e descreve o fluxo real.

A5. Backup da limpeza histórica existe (caminho + nº de registos).

ENTREGA: hash(es) de commit + push, outputs de T1-T3 e A1-A4, lista de registos [TESTE] criados. PROIBIDO tocar em newsletter_subscriptions e em qualquer envio que não seja para ruimedalha@hotelequip.pt.

PROMPT 2 — Identificação: email e WhatsApp ligam-se à ficha de cliente

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07):

- email_threads: só 9 de 319 têm contact_id. conversations (WhatsApp): 0 de 213 têm contact_id — o customer_name é o número cru (ex.: 351962172009).
- contacts tem os campos: email, contact_email, email_compras, email_comercial, email_encomendas, phone, contact_phone, mobile_phone, whatsapp_number (usa TODOS no matching).
- leads tem dedupe_key, contact_id, e a promoção lead→contacto já existe na página de Leads.

OBJETIVO: qualquer email ou WhatsApp que entre fica automaticamente ligado ao contacto certo (e visível na Ficha 360). Sem correspondência → segue como lead (regra atual, não mexer).

REGRAS DE MATCHING (implementar como função única partilhada, server-side):

1. Normalização de telefone PT: remover espaços/hífen; aceitar +351/00351/351/nada → comparar pelos últimos 9 dígitos.
2. Ordem de match: email exato (qualquer dos campos de email de contacts) → telefone normalizado (qualquer dos campos de telefone/whatsapp) → NIF se presente no texto.
3. Match encontrado → escreve contact_id na thread/conversa/lead E preenche customer_name da conversa com o nome do contacto (company_name ou contact_name).
4. Ambiguidade (2+ contactos) → NÃO liga; marca needs_review=true (cria o campo se necessário, via psql) e regista os candidatos numa nota.

TAREFAS:

1. Função de matching no ponto de ingestão de email E de WhatsApp (descobre onde cada canal grava – documenta em docs/fluxo-identificacao.md antes de mexer). ANTES DE ESCREVER DE NOVO: avalia importar/adaptar do repo RuiMedalha/crm-lab-dev (branch feat/crm-realtime-propostas-push) os hooks useResolvedContactId e use-contact-by-id, já analisados e marcados para portar – importa só esses ficheiros, adaptados aos nossos paths, sem trazer mais nada da branch.
2. BACKFILL histórico com backup prévio: corre o matching sobre as 319 threads e as 213 conversas whatsapp existentes. Reporta: quantas ligadas por email, por telefone, ambíguas, sem match.
3. Garante que a Ficha 360 (Customer360Shell) mostra as conversas/threads ligadas – o painel ComunicacoesCliente360Panel já existe; verifica que puxa por contact_id e corrige se necessário (correção mínima, sem refactor).
4. FICHA DE LEAD ATIVA (timeline): via psql, acrescenta lead_id (nullable) a email_threads e a conversations; na ingestão, quando o remetente corresponde a uma lead (email/telefone, mesma normalização) e não a um contacto, liga a thread/conversa à lead. No detalhe da lead (página Leads), mostra: timeline (emails, chamadas Telecof, WhatsApp ligados), follow-ups da lead (acrescenta follow_ups.lead_id via psql se não existir) e produtos pedidos (lead_data.itens). UI mínima, sem refactor da página.
5. Schema: qualquer campo novo via docker exec db-hotelequip psql + POST /utils/cache/clear + sleep 3.

TESTES REAIS OBRIGATÓRIOS:

- T1. Confirma que existe (ou cria) um contacto [TESTE] com email ruimedalha@hotelequip.pt e whatsapp_number 916542271.
- T2. Email: envia de ruimedalha@hotelequip.pt para a caixa ingerida um email [TESTE]. VERIFICA: a thread fica com contact_id do contacto [TESTE].
- T3. WhatsApp: envia via Evolution API (instância ativa) uma mensagem [TESTE] PARA 916542271. Quando a conversa aparecer em conversations, VERIFICA: contact_id preenchido e customer_name = nome do contacto (não o número).
- T4. Ficha 360 do contacto [TESTE]: abre em produção e confirma que o email e a conversa aparecem (screenshot ou output da API que o painel usa).
- T5. Timeline da lead: cria uma lead [TESTE] com email lead.teste@exemplo.pt; simula (registo direto marcado [TESTE]) uma thread com esse remetente. VERIFICA: email_threads.lead_id preenchido e a timeline aparece no detalhe da lead com follow-up de teste associado.

CRITÉRIOS DE ACEITAÇÃO (outputs reais):

- A1. % de threads com contact_id antes vs depois do backfill (query agregada).
- A2. % de conversas whatsapp com contact_id antes vs depois.
- A3. Zero ligações erradas na amostra: mostra 5 exemplos ligados com o par (email/telefone da conversa ↔ campo do contacto que fez o match).

A4. docs/fluxo-identificacao.md no repo.

ENTREGA: hash(es) de commit + push, outputs T1-T4 e A1-A4, caminho do backup, lista de registos [TESTE]. PROIBIDO: mensagens para qualquer número ≠ 916542271; emails para qualquer endereço ≠ ruimedalha@hotelequip.pt; tocar na newsletter.

PROMPT 3 — Anexos e produtos: do email à proposta pré-preenchida

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07) — CASOS REAIS PARA USAR COMO TESTE:

- CERCIESTA: thread 18374172-852a-4d74-8d2c-2106c7b7a3a6, mensagem com attachments JSON: [{file:"496d6894-d741-44ff-bc56-6219c1c28c30", filename:"equip hoteleiro.pdf"}]. É um Excel impresso a PDF com 10 equipamentos (balde pensos slim, banca p/ descascadora Sammic PP6, banho-maria bancada 4-5 GN, carro inox talheres, carro limpeza, carro roupa, lavadora pavimentos s/ fios, micro-ondas 25L inox, tostadeira dupla ≥3,6kW, armário medicamentos).
- INSE: thread f61c5c3f-270b-4bbb-bdda-085a3870b88d, SEM anexo, produto no corpo: "Sammic UX-100" (máquina lavar loiça, 300 refeições/dia).
- Os anexos vivem no campo JSON email_messages.attachments (file = UUID em directus_files). A coleção email_attachments está vazia/morta — NÃO a uses.
- Matching testado no Meilisearch (products_palamenta): "banho maria bancada" → VTVS9040BMGT 1574€; "tostadeira grelhador contacto" → MS2.0.068.0000 511€; "microondas inox 25" → EU1708 344€; "carro inox prateleiras" → MS2006.412.002 158€. Nem tudo faz match ("banca descascadora" falhou) — por isso existe needs_review.

OBJETIVO: quando chega um pedido_orcamento, o sistema extrai os produtos (do corpo E dos anexos) e cria uma proposta em RASCUNHO com cliente + itens pré-carregados. O agente revê e completa. NUNCA envia sozinho.

PIPELINE A IMPLEMENTAR (n8n e/ou extensão Directus — usa a infra existente):

1. Trigger: thread com category=pedido_orcamento (novo, do Prompt 1).
2. Recolha de conteúdo: body_text + cada anexo do JSON attachments:
 - .xlsx/.csv → parse direto das linhas
 - .pdf com texto → extração de texto
 - .pdf digitalizado / imagens → visão via proxy ai (envia o ficheiro em base64 ao ai-proxy)

3. Extração estruturada via proxy ai: devolve JSON {cliente: {nome,email,telefone,nif,morada}, itens: [{descricao,quantidade,referencia,specs}], urgencia, prazo}. Pede APENAS JSON no prompt do modelo.
4. Matching de produtos no Meilisearch (DIRETO, índice products_palamenta): 1º por sku/ean se houver referência; 2º por título. Score baixo ou 0 hits → item fica needs_review=true sem produto associado.
5. Criação: quotation em status=draft + quotation_items (matched: com sku/preço do catálogo; não-matched: só descrição + needs_review). Prefill do cliente: se a thread tem contact_id (Prompt 2) usa o contacto; se tem lead_id, usa os dados da lead. Liga quotation à thread (email_threads.quotation_id) e à lead (quotations.lead_id via psql, se necessário).
6. REGRA DE PROMOÇÃO (decisão do Rui): quando uma proposta é criada a partir de uma lead (pipeline OU manual), a lead é promovida automaticamente a contacto usando o fluxo de promoção existente – todos os dados da lead (nome, empresa, telefone, NIF, morada, website, sku_history) povoam a ficha do contacto, a lead fica status=converted, e o contacto mostra badge visível "Convertida de lead" (usa entity_status ou tag). A quotation e o deal ficam ligados ao contacto novo. Nada disto toca nos campos newsletter_* do contacto.
7. Notificação interna: cria follow_up/atividade "Proposta pré-preparada para rever" atribuída ao agente (por agora, sem atribuição automática, deixa por atribuir).
8. NOTA: usa o helper n() para números do Directus e nunca metas chaves no frontend.

INVESTIGAÇÃO OBRIGATÓRIA INCLUÍDA: as 6 propostas PRP de julho estão em draft com cliente e total NULL. Descobre porquê (autosave a gravar antes do StepClient? campos não persistidos?) e corrige – a proposta criada pelo pipeline E a criada manualmente têm de guardar customer_id/customer_name desde o passo 0. Documenta a causa em docs/propostas-drafts-vazios.md.

TESTES REAIS OBRIGATÓRIOS:

- T1. Corre o pipeline sobre a thread CERCIESTA real. VERIFICA: quotation [TESTE] draft criada com ≥8 quotation_items, dos quais ≥4 com match de catálogo (mostra sku+preço) e os restantes needs_review.
- T2. Corre sobre a thread INSE real. VERIFICA: item "Sammic UX-100" presente; se não existir no catálogo, needs_review=true com a referência guardada.
- T3. Abre a proposta [TESTE] no frontend (rota /propostas/:id) e confirma que os itens e o cliente aparecem no formulário de 8 passos sem crash.
- T4. Envio de teste: envia a proposta T1 APENAS para ruimedalha@hotelequip.pt. VERIFICA: status=sent, email recebido, link público /p/:token abre.
- T5. Promoção: cria uma lead [TESTE] completa (com telefone, cidade, produtos

em lead_data) e gera proposta a partir dela. VERIFICA: contacto criado com todos os dados da lead, badge "Convertida de lead" visível, lead status=converted, quotation ligada ao contacto novo, e campos newsletter_* do contacto vazios/intactos.

CRITÉRIOS DE ACEITAÇÃO (outputs reais):

- A1. JSON da extração da CERCIESTA (os 10 itens reconhecidos).
- A2. Tabela item→match (sku, preço, score) ou needs_review.
- A3. Query: quotations de hoje com customer_id NOT NULL = 100% das criadas pelo pipeline.
- A4. Causa dos drafts vazios documentada + fix commitado + prova (nova proposta manual grava cliente no passo 0).

ENTREGA: hash(es) de commit + push, outputs T1-T4 e A1-A4, lista de registos [TESTE]. PROIBIDO: enviar propostas/emails a qualquer endereço ≠ ruimedalha@hotelequip.pt; tocar na newsletter; usar o índice products_stage (está errado – só products_palamenta).

PROMPT 4 — Pipeline obrigatório: nenhuma proposta sem deal

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07):

- deals: 11 registos, SEM campo stage (só status texto livre), deal_items com 0 registos. quotations já tem deal_id (quase sempre NULL).
- leads: a promoção lead→contacto já existe. Falta lead→deal.
- Pipeline.tsx existe (Kanban) mas sem substância por baixo.

OBJETIVO: o Pipeline passa a ser o ecrã de trabalho. Toda a proposta pertence a um deal; todo o deal tem fase.

TAREFAS:

1. Schema via docker exec db-hotelequip psql (+ cache clear + sleep 3):
 - deals.stage: enum/string com valores: novo, qualificado, proposta_enviada, negociacao, ganho, perdido. Default: novo.
 - deals.lead_id (integer, nullable) para rastrear a origem.
 - Migra os 11 deals existentes: status atual → stage mais próximo (documenta o mapeamento).
2. Conversão lead→deal: na página de Leads, ação "Criar deal" (um clique) que cria deal (stage=qualificado, customer_id se a lead tem contact_id, title =

nome da lead) e marca a lead como converted. NÃO alterar o fluxo de promoção a contacto existente.

3. Regra "nenhuma proposta sem deal": ao criar quotation (pipeline do Prompt 3 E formulário manual), se não houver deal_id → cria deal automaticamente (stage=qualificado) e associa. Ao enviar a proposta → deal.stage=proposta_enviada. Aprovada → ganho. Rejeitada → perdido.

4. Kanban (Pipeline.tsx): colunas = stages; cartão mostra título, cliente, valor (usa helper n()), dias na fase; arrastar entre colunas grava stage. Correções mínimas ao que existe – sem refactor.

5. Backfill: para as 58 quotations existentes sem deal_id, cria deals retroativos (com backup prévio) com stage derivado do status da quotation (draft→qualificado, sent/viewed→proposta_enviada, approved→ganho, rejected→perdido).

TESTES REAIS OBRIGATÓRIOS:

T1. Converte a lead [TESTE] (ou a lead INSE real, sem lhe enviar nada) em deal com um clique. VERIFICA: deal criado, stage=qualificado, lead marcada converted.

T2. Cria uma proposta [TESTE] manual sem deal. VERIFICA: deal auto-criado e associado.

T3. Envia a proposta [TESTE] para ruimedalha@hotelequip.pt. VERIFICA: deal.stage=proposta_enviada. Aprova pelo link público. VERIFICA: stage=ganho.

T4. Abre o Kanban em produção: as colunas mostram os deals do backfill; arrasta um deal [TESTE] e confirma que o stage persiste (query antes/depois).

CRITÉRIOS DE ACEITAÇÃO (outputs reais):

A1. Query: 100% das quotations com deal_id NOT NULL após backfill.

A2. Query agregada deals por stage (o funil tem números reais).

A3. Mapeamento da migração dos 11 deals antigos documentado em docs/pipeline.md.

ENTREGA: hash(es) de commit + push, outputs T1-T4 e A1-A3, caminho do backup, lista de registos [TESTE]. PROIBIDO: newsletter; envios fora de ruimedalha@hotelequip.pt / 916542271.

PROMPT 5 — Seguimento automático + resposta preparada a pedido ("Aceitar e enviar")

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07):

- quotations JÁ tem: viewed_at, last_viewed_at, view_count, follow_up_1_sent_at, follow_up_2_sent_at, follow_up_3_sent_at, follow_up_at, follow_up_notes. Os webhooks n8n quotation-sent/approved/rejected existem no .env.
- email_threads JÁ tem: ai_draft, draft_message_id, sla_due_at, first_replied_at.
- follow_ups: 44 abertos / 3 concluídos – existe a coleção, falta a disciplina automática.
- REGRA DO RUI: a resposta por IA só é preparada QUANDO O AGENTE PEDE (nunca automática para todos os emails), e tem de poder ser enviada de imediato com um clique.

OBJETIVO A – Follow-up automático de propostas:

1. Workflow n8n agendado (2x/dia): propostas com status sent/viewed, sem approved/rejected, cuja última interação (sent_at ou last_viewed_at) tem $\geq 72h$ e follow_up_1_sent_at IS NULL $\rightarrow$ cria follow_up na Agenda do agente do deal (ou por atribuir) com contexto ("Proposta X vista há 3 dias, sem resposta") e marca follow_up_1_sent_at=now(). Segundo ciclo: +5 dias $\rightarrow$ follow_up_2. Terceiro: +7 $\rightarrow$ follow_up_3 com sugestão de fecho.
2. O follow-up criado aparece na Agenda (verifica a página Agenda.tsx e liga se necessário, correção mínima).
3. IMPORTANTE: este workflow NÃO envia nada ao cliente – cria tarefas internas. Envio ao cliente é sempre decisão do agente.

OBJETIVO B – "Preparar resposta" + "Aceitar e enviar" (email):

1. No detalhe do email (EmailThreadDetail.tsx): botão "Preparar resposta". Ao clicar $\rightarrow$ chama endpoint server-side que junta: corpo do email, ai_summary, histórico do contacto (se contact_id), e itens/proposta ligada (se quotation_id) $\rightarrow$ proxy ai gera rascunho em português $\rightarrow$ grava em email_threads.ai_draft e mostra ao agente em editor.
2. Botões no rascunho: "Aceitar e enviar" (envia JÁ via Graph, como resposta na thread, e marca first_replied_at) | "Editar" (edita antes de enviar) | "Descartar".
3. Sem geração automática em background para todas as threads – só a pedido. (Poupa tokens e ruído.)
4. Assinatura: usa email_signature_html do utilizador (bug do mojibake já foi corrigido – confirma que a assinatura sai limpa).
5. SEGURANÇA: qualquer HTML apresentado ao agente ou enviado (ai_draft, assinatura) passa por DOMPurify – o pacote JÁ está no package.json mas nunca foi usado; segue o padrão allowlist do commit f39cbdea do crm-lab-dev (sanitizador na proposta pública do Mark) como referência.

TESTES REAIS OBRIGATÓRIOS:

T1. Força uma proposta [TESTE] com sent_at há 4 dias (update direto com nota [TESTE]). Corre o workflow. VERIFICA: follow_up criado, visível na Agenda, follow_up_1_sent_at preenchido, e NENHUM email saiu para o cliente.
T2. Na thread [TESTE] (a do Prompt 2, ligada a ruimedalha@hotelequip.pt): clica "Preparar resposta". VERIFICA: ai_draft gravado e apresentado.
T3. Clica "Aceitar e enviar". VERIFICA: email chega a ruimedalha@hotelequip.pt como resposta na mesma thread, com assinatura limpa (sem mojibake), first_replied_at preenchido.
T4. Query de segurança: nenhum email enviado hoje pelo sistema tem destinatário ≠ ruimedalha@hotelequip.pt.

CRITÉRIOS DE ACEITAÇÃO (outputs reais):

A1. Lista dos follow-ups criados pelo workflow (query) – todos internos.
A2. Screenshot/HTML do rascunho gerado + prova do envio (message id do Graph).
A3. n8n: alterações a Code nodes seguem o protocolo PUT → desativar → sleep 5 → ativar.

ENTREGA: hash(es) de commit + push, outputs T1-T4 e A1-A3, lista de registos [TESTE]. PROIBIDO: newsletter; qualquer envio fora de ruimedalha@hotelequip.pt.

PROMPT 6 — Pesquisa de produtos nos canais + respostas rápidas que crescem

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

CONTEXTO VERIFICADO EM PRODUÇÃO (15/07):

- Meiliseach responde em <https://search.palamenta.com.pt>, índice products_palamenta, com title, sku, price, stock_status, thumbnail, url (testado: "microondas inox 25" → EU1708 344,25€ instock).
- message_templates existe com 6 registos.
- Espaços de trabalho onde o agente responde a clientes: conversa WhatsApp (Comunicações), chat do site (askme) e TelecofCallWorkspace.tsx (chamadas).

OBJETIVO: em qualquer conversa (WhatsApp, chat, chamada Telecof), o agente pesquisa produtos e responde em segundos; as respostas boas ficam guardadas e a lista cresce com o uso.

TAREFAS:

1. Componente partilhado ProductSearchPanel (um só, em src/components/products/): input de pesquisa → Meilisearch DIRETO via fetch (VITE_MEILISEARCH_*, índice products_palamenta) → cartões com thumbnail (objectFit: contain, NUNCA cover), título, sku, preço (helper n()), stock, e botões: "Inserir na conversa" (texto com nome+preço+link) e "Copiar link".
2. Integra o painel nos 3 espaços: conversa WhatsApp, chat do site, TelecofCallWorkspace (neste último, painel lateral – o agente está ao telefone e lê em voz alta; o botão inserir prepara mensagem WhatsApp para enviar ao cliente SE a conversa existir).
3. Respostas rápidas:
 - via psql: message_templates.usage_count (integer, default 0) e message_templates.channel (string, nullable).
 - Botão "Respostas rápidas" na caixa de escrita das conversas: lista ordenada por usage_count desc, filtro por texto; ao usar, incrementa usage_count.
 - Depois de o agente enviar uma mensagem escrita à mão: botão discreto "Guardar como resposta rápida" (título + texto pré-preenchidos).
 - Variáveis simples nos templates: {{nome}}, {{produto}}, {{preço}} substituídas ao inserir.
4. Sem refactor dos espaços de trabalho – o painel entra como componente adicional.

TESTES REAIS OBRIGATÓRIOS:

- T1. Na conversa WhatsApp [TESTE] com 916542271: pesquisa "micro-ondas", insere o cartão do EU1708 e envia. VERIFICA: mensagem chega ao 916542271 com nome, preço e link corretos.
- T2. Guarda essa mensagem como resposta rápida [TESTE]. VERIFICA: aparece na lista, usage_count incrementa ao reutilizar.
- T3. Abre o TelecofCallWorkspace em produção e confirma o painel de pesquisa a funcionar (screenshot ou verificação da chamada Meilisearch no network).
- T4. Chat do site: mesma verificação do painel na conversa askme.

CRITÉRIOS DE ACEITAÇÃO (outputs reais):

- A1. Um só componente ProductSearchPanel importado nos 3 sítios (grep no repo como prova).
- A2. Zero chamadas ao Meilisearch via proxy Directus (grep por product-search = sem novos usos).
- A3. Bundle: confirma que o build não cresceu descontroladamente (tamanho antes/depois).

ENTREGA: hash(es) de commit + push, outputs T1-T4 e A1-A3, lista de registos [TESTE]. PROIBIDO: mensagens para números ≠ 916542271; newsletter intocável.

PROMPT 7 — Auditoria contínua: o script que mede tudo

Lê primeiro o CLAUDE.md (incluindo a secção SPRINT FINAL) e cumpre-o integralmente.

OBJETIVO: um script único que o Rui corre a qualquer momento para ver a saúde do CRM — os mesmos números da auditoria de 15/07, comparáveis no tempo.

TAREFAS:

1. Cria scripts/audit.sh no repo (usa curl + python, token Directus por variável de ambiente DIRECTUS_TOKEN, nunca hardcoded) que imprime:
 - Funil: contactos | leads (por status e por source) | deals (por stage) | quotations (por status) | ganhas
 - Ligações: % email_threads com contact_id | % conversas whatsapp com contact_id
 - Triagem: threads de hoje por category+status | leads criadas hoje por source
 - Propostas: criadas hoje | com deal_id (%) | com customer_id (%) | vistas sem resposta >72h
 - Follow-ups: abertos | concluídos | criados hoje
 - Alertas: leads incoming >48h sem dono | propostas needs_review por rever
2. Output em texto limpo (tabela simples) E opção --json para consumo futuro no Dashboard.
3. Documenta no README da pasta scripts/ como correr (incluindo export do token).
4. Corre o script AGORA e inclui o output completo no relatório — é a linha de base pós-sprint.

CRITÉRIOS DE ACEITAÇÃO:

- A1. Script no repo, corre sem erros, sem tokens hardcoded.
- A2. Output real incluído no relatório (linha de base).

ENTREGA: hash de commit + push + output do script. Este output é comparado com a auditoria de 15/07/2026 (leads incoming: 1.055 | WA ligadas: 0/213 | threads ligadas: 9/319 | propostas com deal: ~0% | follow-ups concluídos: 3/52).

DEPOIS DO SPRINT (não incluído nestes prompts — decidir depois)

- Ligação WooCommerce (encomendas + carrinhos abandonados →

Pedidos/Carrinhos) — "é só puxar com ligação", fase própria.

- Chat do site ao vivo (handoff askme → agente; a infra mode/ai_enabled/handoff_rules já existe) — **importar do crm-lab-dev**: realtimeMessages.ts (WebSocket) e siteChat.ts da branch feat/crm-realtime-propostas-push, em vez de construir de raiz.
- Fila "Novidades" → gerador de publicações/blog (os emails fornecedor_novidade do Prompt 1 já ficam guardados).
- Limpeza das 978 leads "central" (decisão D4 do documento — fazer com backup, fora deste conjunto de prompts para não misturar).
- Módulo pós-venda/reclamações.

ANEXO — IMPORTS DO crm-lab-dev (validado a 15/07 contra o estado real dos dois repos)

Já importado a 02/07 (não repetir): fix CORS, inbox mobile drill-down, PWA/Web Push (push_subscriptions), infra de social posting.

A importar dentro deste sprint (já embutido nos prompts):

Item do crm-lab-dev	Branch	Encaixa em
Hooks useResolvedContactId + use-contact-by-id	feat/crm-realtime-propostas-push	P2 (identificação)
Padrão sanitizador allowlist XSS (commit f39cbdea)	feat/crm-realtime-propostas-push	P5 (render do ai_draft; dompurify já instalado e nunca usado)

A importar depois do sprint:

Item	Branch	Destino
realtimeMessages.ts (WebSocket) + siteChat.ts + NotificationBell	feat/crm-realtime-propostas-push / feat/crm-bell-mobile-ui	Chat do site ao vivo (fase pós-sprint)
Página /relatorios (funil, receita)	feat/crm-bell-mobile-ui (commits 09/07)	UI do Dashboard dos 7 KPIs, alimentada pelo audit.sh do P7
Ações em massa de contactos (selecionar, tag, atribuir)	feat/crm-bell-mobile-ui	Página de Contactos, pós-sprint

NÃO importar: o módulo de propostas paralelo do Mark (PropostaPublica.tsx e afins) — o nosso módulo de 8 passos é o oficial (decisão D2/P3); trazer o dele criaria um terceiro sistema de propostas. A nossa página pública atual não injeta HTML (verificado: única ocorrência de dangerouslySetInnerHTML é o chart.tsx do shadcn), por isso o fix XSS aplica-se como padrão preventivo no P5, não como correção urgente.